

APIs

¿Podrías explicar con tus
palabras que es una API?
¡Escríbelo en el chat!



Recursos asincrónicos

¿Revisaste los recursos de la semana 6?

- *Presentación de clases*
- *Guía - APIs*
- *Prueba – Conversor de monedas nacionales*

¿Tienes dudas sobre el material o algo revisado en ellos?



/*APIs Rest*/

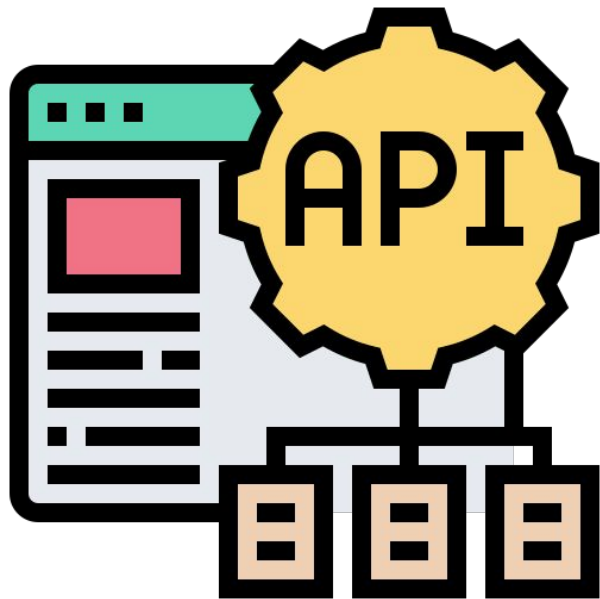
¿Qué es una API?

Es un conjunto de reglas y herramientas que permite que diferentes aplicaciones se comuniquen entre sí.

En términos más simples, una API es como un intermediario que permite que dos programas se hablen entre sí. Proporciona un conjunto de reglas y definiciones que especifican cómo los componentes de software deben interactuar. Esto puede incluir cómo solicitar ciertos datos o funciones de un programa a otro.



Utilidades de las APIs



Hay APIs para distintos propósitos:

- Datos del clima
- Servicios de geolocalización
- Indicadores financieros
- Actualizar redes sociales
- Automatización de acciones en campañas de marketing

Adicionalmente, es relativamente sencillo crear una API cuando tienes nociones de backend.

Cliente servidor



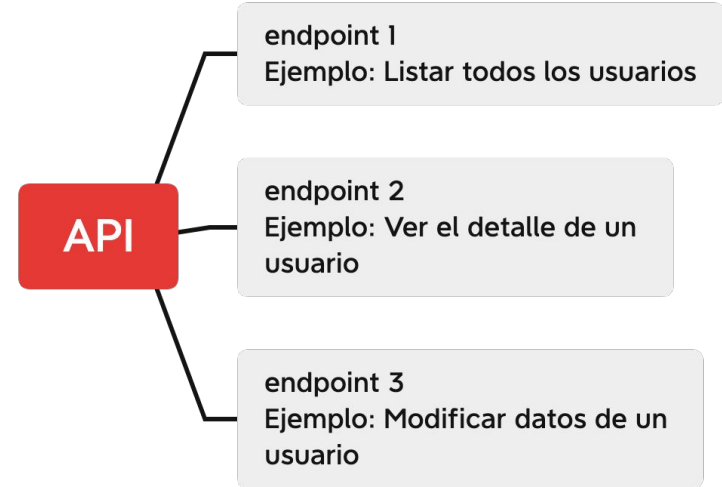
El modelo cliente-servidor es una arquitectura de software donde una aplicación o proceso, conocido como cliente, solicita servicios o recursos de otra aplicación o proceso, conocido como servidor.

Esta arquitectura se utiliza comúnmente en sistemas distribuidos y redes, permitiendo la separación de responsabilidades entre el cliente y el servidor.



Endpoints

Un endpoint es simplemente la URL o dirección que se utiliza para acceder a un servicio o recurso específico proporcionado por una API. Al hacer una solicitud a un endpoint particular, el cliente de la API está indicando qué acción o información específica está buscando.



**/* Obteniendo los datos desde
Javascript */**

Haciendo un request con fetch

Para conectarnos a la API de randomUser desde JavaScript crearemos una página web nueva y dentro de script agregaremos lo siguiente.

```
async function getRandomUser(){  
  const res = await fetch("https://randomuser.me/api")  
  const data = await res.json()  
  console.log(data);  
}  
  
getRandomUser()
```

Viendo la respuesta en la consola del navegador

Si ejecutamos el código anterior podremos observar como se muestra la información por consola:



```
>> ▶ async function getRandomUser(){  
    const res = await fetch("https://randomuser.me/api")  
    const data = await res.json()  
    console.log(data);  
}...  
  
← ▶ Promise { <state>: "pending" }  
  
▼ Object { results: (1) [...], info: {...} }  
  ▶ info: Object { seed: "791d20dd5b4b0de0", results: 1, page: 1, ... }  
  ▼ results: Array [ {...} ]  
    ▶ 0: Object { gender: "female", email: "elena.blanchard@example.com", phone:  
      "02-83-25-28-48", ... }  
      length: 1  
    ▶ <prototype>: Array []  
    ▶ <prototype>: Object { ... }
```

Revisando el código

- Por ahora ignoremos todo lo que diga `async` y `await`.
- `Fetch` hace un request al endpoint.
- `res.json` Transforma los resultados para que podamos leerlos fácilmente como un objeto de Javascript.
- Si quisiéramos consultar otra API, solo tendríamos que cambiar el endpoint (y el nombre de la función para ser coherentes).

```
async function getRandomUser(){  
  const res = await  
  fetch("https://randomuser.me/api")  
  const data = await res.json()  
  console.log(data);  
}  
  
getRandomUser()
```

Revisando el código

```
async function getRandomUser(){  
  const res = await fetch("https://randomuser.me/api")  
  const data = await res.json()  
  console.log(data);  
}
```

getRandomUser()

Se obtiene el contenido en formato JSON de la respuesta

Se obtiene la respuesta(res) de la consulta

Async y Await

`async` y `await` son conceptos que están relacionados con la programación asíncrona en muchos lenguajes de programación, pero en particular se han vuelto muy relevantes en el contexto de JavaScript y otros lenguajes que se ejecutan en entornos de navegador.

1. `async` es una palabra clave que se utiliza para declarar que una función devolverá una Promesa. Una función asíncrona siempre devuelve una Promesa, incluso si no se utiliza explícitamente la palabra clave `return`.
2. `await` es utilizado dentro de funciones declaradas como `async`. Indica que la ejecución de la función asíncrona debe esperar a que la Promesa a la que se está esperando se resuelva antes de continuar. `await` solo puede ser utilizado dentro de funciones `async`.

/* Sentencia Try Catch */

¿Qué es try y catch?

Las sentencias try y catch son parte de la estructura de control de manejo de errores en muchos lenguajes de programación, incluyendo JavaScript. Permiten a los desarrolladores gestionar y manejar errores de manera más controlada en sus programas.

- La sentencia try se utiliza para envolver un bloque de código en el cual se espera que pueda ocurrir algún tipo de error. Dentro del bloque try, se coloca el código que se desea ejecutar. Si ocurre un error en este bloque, el control del programa se transfiere automáticamente al bloque catch.
- La sentencia catch se utiliza para manejar el error que ocurrió en el bloque try. Dentro del bloque catch, se especifica el tipo de error que se está capturando y el código que se ejecutará en respuesta a ese error.

Revisemos el Desafío: "Conversor de monedas nacionales"



{desafío}
latam_

*Academia de
talentos digitales*

